The bench harness

Three gates, each roughly ten times the cost of the one before it. Run them in order; a cheap failure stops an expensive one.

gate	cost	needs	what it answers
`tools/release_sweep.py --offline`	~1 min	nothing	can this code possibly work
`tools/bench_smoke.py`	11.4 min	both instruments	is the bench and the harness sound
`tools/soak.py --suites formats,plan`	~3.0 h a lap	both instruments	does it hold up over hours

Measured, not estimated: **6.5 s per cell**. A lap of 39 waveforms at 43 rates each is 1677 cells, so **about 3.0 hours**; all 41 waveforms is 3.2 h. An offline lap of the same cells is **18 seconds**.

Eight hours is therefore between two and three laps. That is the floor, not a target: the point of the soak is a failure rate per point, and two laps can only ever say "twice" or "once" or "never".

The seeded sweep

`tools/soakplan.py` decides what one iteration tests. Everything comes from the iteration number, so **iteration 129 is reachable without running 128 first**.

Per iteration, three things vary and nothing else does:

- the **order** waveforms load in, so a state leak out of one stops looking like a defect in whichever waveform always follows it
- the **non-standard rate** drawn in each gap between standard rates
- the **wait** before each capture, per cell

Fixed: the 22 standard baud rates in `[300, 250000]`, tested on every waveform every iteration. `250000` is `sdec.maxbaud`; below 300 costs 8 s of capture for a rate nothing here speaks.

Frame mode only. Above 165563 Bd `sdec.fs_for_burst` returns nil, so the streaming paths cannot record 172800 and up at all, and only frame reaches 250000.

Why the wait is scaled, not constant

Uniform over **10 byte-times** — which is 100 bit-times — so one continuous draw randomises the byte a capture lands on *and* the bit *and* the phase within the bit, because it is quantised to none of them. Scaled by baud it costs 0.4 min across a whole sweep where a flat 1 s costs 14.7.

It also lands where it is needed. Below about 20 kBd, socket jitter is smaller than a byte time and cannot move the byte offset at all; above it, jitter alone already scatters the phase by tens of bytes.

Why a uniform draw is not the only rule

`sdec.pick_fs` snaps **up** through a coarse ladder, so below 125 kBd samples-per-bit is a sawtooth whose minimum in each step sits at `baud = floor(fs/8)` — 312, 625, 1250 … 80000, 125000. Those are where the decoder is closest to being confidently wrong, and a uniform draw lands on one with probability about zero. A gap containing an edge therefore yields that edge one time in three. Above 125 kBd fs is pinned at 1 MSa/s and samples-per-bit falls smoothly to 4.00, so there is no cliff and uniform is right.

The smoke gate

Four waveforms, then the button matrix.

```
standard rates   v77  the fox, 8N1, 133 B repeating
                 r06  random,  7E1, 256 B non-repeating
drawn rates      v78  the fox, 7E1
                 r00  random,  8N1
```

Paired crosswise on purpose. Each rate family faces both formats and both content classes. Pairing by family instead — fox on the standard ladder, random on the drawn rates — would leave the standard rates never seeing a non-repeating payload, and that is where head alignment and the parity vote get tested.

86 cells in 9.5 min, then 45 presses in 1.9 min. `--plan-spec vector:std|nonstd|all` is what makes the quarter-coverage expressible; a cell's wait is still keyed on its index in the **full** rate list, so a cell reached this way is the same cell the soak runs.

The receipt

A pass writes `out/smoke-receipt.json` holding a hash of `tsp/` and `tools/`. `tools/hooks/pre-push` recomputes that hash and refuses a push when either tree has moved, so "I ran the smoke test" becomes a claim about the exact code being pushed rather than about the past. The hash folds in the working tree, not just the index, so an unstaged edit invalidates it too.

`SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason. Docs-only pushes need no receipt.

Changing your mind mid-run

`--hours` and `--laps` are both decided before the first lap, so "make that four laps, not three" would otherwise cost a restart — and a restart throws away the laps already banked, which is the opposite of what one-more-lap means. Two files in the record directory are read before each lap:

```
echo 4 > <record dir>/LAPS      # run exactly this many laps, then stop
touch  <record dir>/STOP        # stop cleanly after the lap in flight
```

Neither interrupts a lap. **A lap is the unit of evidence** — a truncated one is not tallied, so cutting one wastes every minute already spent on it. For the same reason the wall deadline from `--hours` only gates *starting* a lap: one that begins inside the budget always runs to completion.

The release gate, stage by stage

`tools/release_sweep.py` runs every check in one go and writes an auditable record to `out/release/<timestamp>/`: a `REPORT.md`, a `summary.json`, every stage's full output, every front-panel screenshot, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails. `--offline` skips every stage needing an instrument and runs in seconds; `--skip name,name` drops named stages, and a skipped gate is reported as *not passed* rather than passed.

The full sweep needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the app refuses a second build in one power cycle, so the sweep checks that up front rather than failing forty minutes in. It also needs the SDG2122X loaded with the stimulus waveforms (`bench_matrix.py --upload`), and only one client may hold the DMM's control socket at a time.

Stage	Checks
`lint` `parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`vecrefs`	every waveform id named in `tools/` is in `MAP` or declared `RETIRED` — deleting from `MAP` alone leaves references that fail where they are used, not where they are declared
`soakrand`	`mt19937.py` and `mt19937.lua` produce one sequence, checked against CPython's `random`, plus the 5.0.2 scan deciding whether the module can load on the instrument
`unit`	1 063 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
`unit-cancel`	one-press recordings, both window sizes, and the flow-control loop with no interaction
`unit-frontrig`	TRIGGER-key and rear-BNC arming *through* `acquire()` — the routing that can drop to free-run while the status row still claims the key is the source
`unit-usblog`	USB log name allocation and the persistent index: exhaustion must refuse rather than loop and hang the panel
`unit-stream`	the streaming arm — both 4915 defences, one press per slice rather than per window
`unit-patterns`	byte-exactness on the hard payloads: edge-density extremes, walking bits and known random data, at the sample rates the panel actually picks
`unit-forcerate`	a forced rate the wire does not carry must **refuse** and name the rate it does carry, with both answers drivable unattended
`unit-judgev`	the harness's three verdicts — too short to read is **inconclusive**, not silently wrong; an alignment survives one bad byte; a loud vector may miss a bounded few. Each with the wrong capture that must still fail
`stress`	hostile signals: never silently **wrong**, never **raises**
`unit-analog`	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the window opens
`unit-phasesweep`	every vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among trustworthy calls
`unit-seam`	a capture whose arb loop seam lands late must still be judged, and the narrower trim still required
`unit-sdgguard`	every route by which an out-of-spec waveform could reach the generator refuses it
`unit-loremgate`	the harness's own long-payload verdict: every clean run validated, flag count bounded
`tolerance`	recomputes the envelope table printed in the manual
`package` `archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: `README.pdf`, `MANUAL.pdf`, `REFERENCE.pdf`, `BENCH.pdf`
`hw-matrix`	through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, three logic swings, twelve DC offsets
`hw-payloads`	fourteen payloads covering every byte value 0–255, two of them also at 115 200 and 250 000
`hw-odd-rates`	nineteen **non-standard** baud rates — 900, 1500, 3600, 8123, 29127, 104857 …

Stage	Checks
`hw-panel`	every button in every state, including a one-press recording. Six checks per press: the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, and **the panel actually shows it** — grabbed before and after each press and differenced by region
`soakrand-dmm`	the third leg: the instrument's **own** Lua 5.0.2 must produce the same words, floats, rejected draws and permutation
`hw-plan`	the seeded sweep on the bench: two waveforms across all 43 rates in a seeded order, with a seeded wait before every capture
`hw-break`	degenerate signals and contradictory settings — no signal, DC only, all-`0x00` / `0xFF` / `0x55`, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, six wrong forced settings. A refusal with a reason passes; confident garbage does not

This table must list every stage `release_sweep.py` defines, or the published inventory of what a release passed is incomplete. The check:

```
import re
code = set(re.findall(r"Stage\(\s*'([^']+)'", open('tools/release_sweep.py').read()))
doc  = {n for l in open('docs/BENCH.md') if l.startswith('| `')
          for n in re.findall(r'`([a-z0-9-]+)`', l.split('|')[1])}
assert not (code - doc), sorted(code - doc)
```

Reproducing a failure

A failing cell prints a `REPRO` line carrying everything needed:

```
REPRO iteration 1 vector v48a rate 2400 Bd (std) srate 24000 spb 10 wait 38.5438 ms
      measured-head - nf 2 fmt 8N1 want 8N1
```

Replay it offline, where there is no analogue path and no race:

```
python3 tools/soakplan.py --emit-lua --iteration 1 > /tmp/p.lua
lua tools/sweep_plan.lua --plan /tmp/p.lua --cell v48a:2400
```

Fails offline too → logic, deterministic, debuggable. **Passes offline** → the difference is the hardware: the DAC, the cable, the digitiser, or the capture phase that the seeded wait can only perturb.

What the seed does and does not reproduce

The seed reproduces the **schedule** — which rates, in which order, with what commanded wait. On hardware it cannot reproduce the realised capture **phase**, because that is a race the wait only disturbs. This is why a failure record carries the **measured** head alignment: that is what converts a hardware failure into an exact offline replay. Offline the phase is set explicitly, so it is exact by construction.

The offline twin

`tools/sweep_plan.lua` replays a plan cell for cell with no instrument: it reads the `.bin` the generator actually holds, converts codewords to volts at the amplitude the file was encoded for, resamples to the rate the app would pick, starts at the phase the seeded wait produces, and decodes.

Interpolation is **linear** between arb points, not a step: a DAC into a reconstruction filter presents slope, and the scope measured clean 0/3.26 V edges. `--hold` is the harsher zero-order-hold model, available as a robustness sweep.

What it is not. It is not the app. `bench_matrix` presses the real Capture button and reads the real panel; this calls `sdec` directly, so it never exercises the two-pass probe, autolock, or anything the operator sees — and it never exercises waveform selection, which is why a selection fault shows up on hardware and passes here. It also judges more simply: the bytes must be a cyclic substring of the payload, where `bench_uart.judge_payload` weighs flag budgets and head damage.

What a waveform is entitled to do

Two classes, in `soakplan.VECTOR_EXPECT`:

- **`exact`** — must decode byte-exact at every rate. Anything else is a defect.
- **`loud`** — may fail, and **must not be silently wrong**. Declining, or failing with flags raised, is a pass. Confident bytes that are not the payload fail for these too.

`loud`: `v47` (spikes stacking to 9.3 V), `v48a` and `v48b` (drift), `j20` (20 % jitter), `v61`–`v63` (LIN carries framing violations no UART frame can contain).

`v48a` is the instructive one. Its 0.6 V drift is inside tolerance **at its native rate**; swept two decades away the drift-to-window ratio changes, the two logic levels stop being the two densest amplitudes, and the app reports `baseline unstable — only 8 % of samples sit at a logic level` and declines. Declining is the correct answer to an unreadable signal, so it counts as one.

Ambiguous framing, and why the oracle accepts two answers

`v90`, `v92` and `v94` are blocks of `00/FF/55/AA` and walking bits. In every one of those byte values **bit 7 carries exactly the parity a 7-bit reading would require**, so the wire is a valid 8N1 frame *and* a valid 7E1/7O1 frame, and nothing in the signal separates them. `v92`'s `exp_hex` is its `.txt` with bit 7 stripped — `80 → 00`, `FE → 7E`.

Which reading the app votes for shifts with the rate, so `v94` reads 8N1 at some rates and 7E1 at others, correctly both times. The judge therefore accepts **either** reading, and the format check stands down where there is more than one. Demanding either single answer fails a correct decode at every rate that chose the other.

The random numbers

`tools/mt19937.py` and `tools/mt19937.lua` must produce one sequence, because the hardware sweep is driven from Python and the offline suites draw in Lua. `tools/test_soakrand.py` holds three legs together:

```
CPython random  ->  mt19937.py  ->  host Lua  ->  the instrument's own Lua 5.0.2
```

CPython's `random` **is** this algorithm, so it is an independent implementation rather than a pasted table of numbers. The Lua side is arithmetic-only on doubles — 5.0.2 has no bitwise operators, no `%` and no `#`, and all three are parse errors there, so one occurrence stops the whole module loading whether the branch is taken or not. `mul32` splits its multiplier so no product exceeds 2^48; a direct `1812433253 * (2^32-1)` is 860× over 2^53 and would silently lose the low bits it carries forward.

Decisions are keyed — `{magic, iteration, purpose, index}` — not drawn from one running stream, so replaying a single cell does not depend on how many ran before it, including any that failed.

Instrument hazards

The generator's LAN service wedges, and there is no way to recover it remotely. No instrument on this bench has a smart plug, so every power cycle — a wedged SDG, the DMM's once-per-power-cycle UI build limit, a DMM control socket left held by a client that died — costs a human at the bench, and an unattended run that wedges is over until someone arrives. Two known triggers for the generator: a few consecutive large `WVDT` uploads, and sweeping SRATE across many rates on a very large stored waveform.

So a per-waveform health check asks both instruments whether they are still answering, using a Lua round trip rather than `*IDN?` — the SCPI parser can keep answering after the app is gone. On a silent instrument `bench_matrix` exits **3**, and `soak.py` ends the whole run on that rather than tallying laps against a dead bench. Exit 1 means "a cell failed" and a soak should carry on from it.

One client each. The DMM6500 accepts one controlling socket and the SDG one SCPI session; a second connection silently steals replies. `sdg_alive` must be passed an already-open handle or it reports a wedge that is not there.

Watching a long run

`soak.py` captures each lap's output rather than streaming it, so a 155-minute lap is otherwise opaque from outside. `bench_matrix --heartbeat FILE` appends a flushed, timestamped line as each waveform starts and finishes, with cells, seconds per cell, amplitude, unexpected count, and whether each instrument answered. A healthy run appends one every ~2.5 min.

Do **not** use the child's CPU time as a liveness signal. The work is I/O bound at about 0.6 s of CPU an hour, and the child's pid changes at every lap boundary with its counter resetting to zero — so a monotonic check reports a stall at precisely the moment a lap completes.